

Programação para internet 1

Javascript

Prof. Luís Eduardo Tenório Silva
luis.silva@garanhuns.ifpe.edu.br

Elementos básicos de uma página Web

2

- **HTML**

- » Linguagem de marcação;
- » Define a estrutura de uma página.



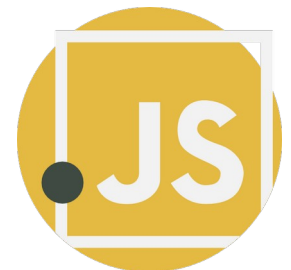
- **CSS**

- » Linguagem de estilização;
- » Define a aparência dos elementos de uma página.



- **Javascript**

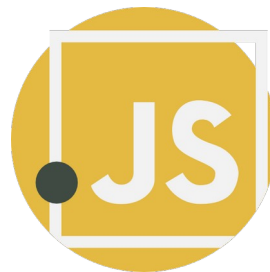
- » Linguagem de programação;
- » Define a interatividade de uma página;
- » Emite e trata eventos disparados pelo usuário.



O que é Javascript?

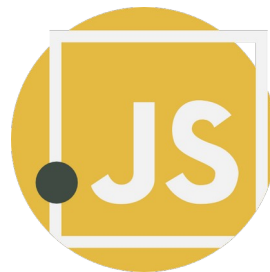
3

- Linguagem de programação que suporta múltiplos paradigmas;
- Utilizada para tornar as páginas Webs interativas:
 - » Animações complexas;
 - » Interação entre usuário e elementos HTML;
- Executada pelo navegador

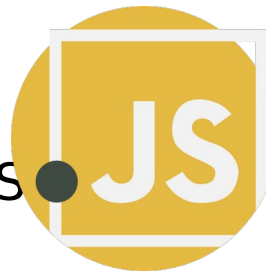


Características da linguagem

- Suporte universal
- Linguagem interpretada
 - » Inexistência de processo de compilação.
- Tipagem dinâmica
 - » O valor de uma variável define o seu tipo.
- Manipulação do DOM
 - » Alterar de forma dinâmica os elementos de uma página Web.
- Assíncrona
 - » Realização de atividades de forma não-sequencial.
- Portável
 - » Multiplataforma, executado em diversos SO através do Browser.



- Antes do Javascript
 - » Páginas estáticas com pouca interatividade;
 - » Uso de **CGI** (*Common Gateway Interface*) para adicionar dinamismo.
- 1995 – Nascimento do JS
 - » Criado por Brendan Eich na Netscape Communications;
 - » Inicialmente chamava-se mocha
 - Renomeada para livescript e, devido o sucesso do java na época, Renomeada novamente para javascript.
- 1996
 - » Microsoft cria uma linguagem com o mesmo intuito (Jscript)
- 1997
 - » Primeiras necessidades de padronização (Surgimento do ECMA



- 2000

- » Evolução da linguagem para adicionar suporte a recursos modernos;
- » Surgimento de bibliotecas como jQuery, Prototype e Dojo.

- 2009

- » Surgimento do NodeJS (execução do JS fora do navegador);

- Atualmente

- » Linguagem de programação predominante na web;
- » Um rico acervo de bibliotecas (React, Vue, Svelte), frameworks (Angular, Next) para desenvolvimento de aplicativos webs modernos.



- Especificação do ECMA Internacional para linguagens de script
 - » Define um **padrão** a linguagem Javascript;
 - » Resolvendo o problema de interoperabilidade.
- Define quais **recursos** a linguagem possui;
- Todos os navegadores modernos aplicam as especificações ECMA;
- Versões do ECMAScript e principais definições:
 - » ES1 (1997): Introduziu os conceitos de variáveis, funções, objetos, estruturas de controle, etc.
 - » ES2 (1998)
 - » ES3 (1999): Adição de tratamento de exceções (try/catch)



» ...

- » ES6 (2015): Adicionou recursos de classes, arrow functions, let e const para declaração de variáveis. Modernização do Javascript.
- » ES7 (2016): Inclusão do símbolo de potenciação (**).
- » ES8 (2017): Adição de ***async/await***.
- » ES9 (2018): Operadores Rest e Spread.
- » ES10 (2019): Adicionado recursos para lidar com Arrays e Strings.
- » ES11 (2020): Adicionado recursos de cadeia opcional (?.) e coalescência (??).
- » ES12 (2021): Tratamento de promises e operadores lógicos encadeados.
- » ES13 (2022): Top level await e campos privados para classes e métodos.

Adicionando JS em uma página Web

- Existem duas maneiras de adicionar código JS em uma página Web:
 - » Elemento `<script>` dentro do elemento `<body>` no arquivo `.html`
 - **Preferencialmente**, como último elemento `<body>`
 - » Elemento `<script>` dentro do elemento `<body>` no arquivo `.html`, com propriedade `src` apontando para um arquivo `.js`

```
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Introdução ao Javascript</title>
7  </head>
8  <body>
9      <p>Hello</p>
10     <script>
11         prompt("Hello world")
12     </script>
13 </body>
14 </html>
```

Adicionando JS em uma página Web

10

- Existem duas maneiras de adicionar código JS em uma página Web:
 - » Elemento `<script>` dentro do elemento `<body>` no arquivo `.html`
 - **Preferencialmente**, como último elemento `<body>`
 - » Elemento `<script>` dentro do elemento `<body>` no arquivo `.html`, com propriedade `src` apontando para um arquivo `.js`

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Introdução ao Javascript</title>
7 </head>
8 <body>
9     <p>Hello</p>
10    <script src="hello.js"></script>
11 </body>
12 </html>
```

Hello World em JS

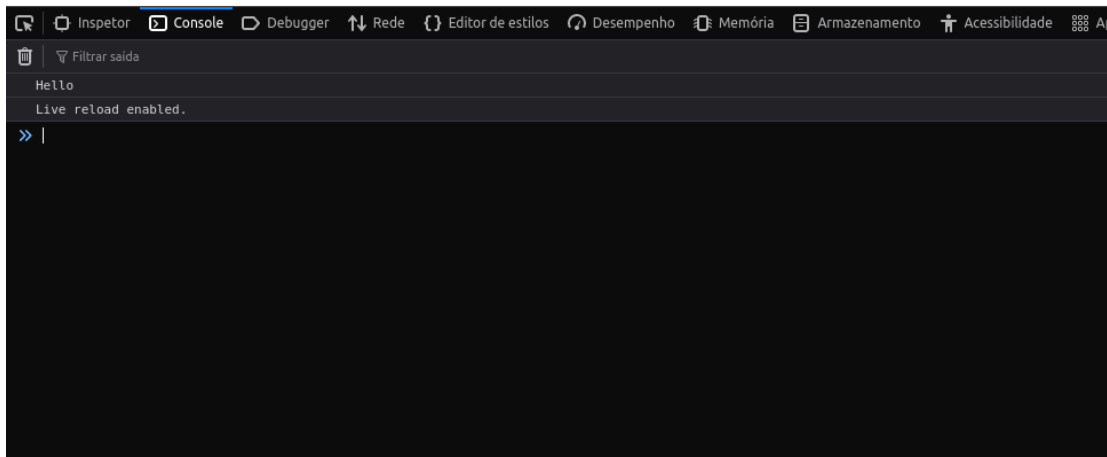
11

- Usando elementos do browser:

alert("Hello World")

- Usando o console Javascript do browser:

console.log("Hello World")



- **var**

- » Possui escopo de função (visível em qualquer escopo);
- » Pode ser redeclarada.

- **let**

- » Possui escopo de bloco (visível apenas dentro do bloco ao qual foi declarado);
- » Não pode ser redeclarada;
- » O valor pode ser redefinido.

- **const**

- » Possui escopo de bloco;
- » Não pode ser redeclarada;
- » O valor não pode ser redefinido.

Tipagem de variáveis

13



```
1  let idade = 30;           // Número inteiro
2  let preco = 19.99;        // Número de ponto flutuante
3  let nome = "Alice";       // String
4  let mensagem = 'Olá, mundo!'; // String
5  let temSol = true;         // Boleano Verdadeiro
6  let estaChovendo = false; // Boleano Falso
7  let x; // Não definido (undefined)
8  let pessoa = null; // Nulo
9
10 // Object
11 let pessoa2 = {
12     nome: "João",
13     idade: 25,
14     cidade: "São Paulo"
15 };
16
17 // Array
18 let frutas = ["maçã", "banana", "laranja"];
19 let divisaoInvalida = 10/0 // Infinity
20 let resultado = Number('numero invalido') // NaN
```



```
1 // Operadores de Atribuição
2 let x = 10;
3 x += 5; // x = x + 5
4 x -= 5 // x = x - 5
```



```
1 // Operadores Aritméticos
2 let a = 10;
3 let b = 3;
4 let soma = a + b; // Soma: 13
5 let subtracao = a - b; // Subtração: 7
6 let multiplicacao = a * b; // Multiplicação: 30
7 let divisao = a / b; // Divisão: 3.333333...
8 let modulo = a % b; // Módulo (resto da divisão): 1
9 let expo = a ** b; // Exponenciação: 1000 (10 elevado a 3)
```



```
1 // Operadores de Comparação
2 let igual = a == b; // Igualdade com coerção de tipo: false
3 let estritamenteIgual = a === b; // Igualdade estrita: false
4 let diferente = a != b; // Desigualdade com coerção de tipo: true
5 let estritamenteDiferente = a !== b; // Desigualdade estrita: true
6 let maiorQue = a > b; // Maior que: true
7 let menorQue = a < b; // Menor que: false
8 let maiorOuIgual = a >= b; // Maior ou igual a: true
9 let menorOuIgual = a <= b; // Menor ou igual a: false
```




```
1 // Operadores Lógicos
2 let condicao1 = true;
3 let condicao2 = false;
4 let resultadoE = condicao1 && condicao2; // E lógico: false
5 let resultadoOU = condicao1 || condicao2; // OU lógico: true
6 let resultadoNao = !condicao1; // NÃO lógico: false
```



```
1 // Operadores de Incremento/Decremento
2 let contador = 5;
3 contador++; // Incremento: 6
4 contador--; // Decremento: 5
```



```
1 // Operadores de Bitwise
2 let numero1 = 5; // Binário: 0101
3 let numero2 = 3; // Binário: 0011
4 let resultadoAND = numero1 & numero2; // AND bit a bit: 0001 (1 em decimal)
5 let resultadoOR = numero1 | numero2; // OR bit a bit: 0111 (7 em decimal)
6 let resultadoXOR = numero1 ^ numero2; // XOR bit a bit: 0110 (6 em decimal)
7 let resultadoNOT = ~numero1; // NOT bit a bit: 1111111111111111111111111111010 (-6 em decimal)
8 let resultadoShiftLeft = numero1 << 1; // Shift à esquerda: 1010 (10 em decimal)
9 let resultadoShiftRight = numero1 >> 1; // Shift à direita com preenchimento de sinal: 0010 (2 em decimal)
10 let resultadoShiftRightZeroFill = numero1 >>> 1; // Shift à direita sem preenchimento de sinal: 0010 (2 em decimal)
11
```



```
1 // Operadores de Concatenação
2 let texto1 = "Olá, ";
3 let texto2 = "mundo!";
4 let mensagem = texto1 + texto2; // Concatenação de strings: "Olá, mundo!"
```

- Controla o fluxo de execução do código
 - » if
 - » if/else
 - » if/else if /else
 - » switch

Estruturas de decisão (if)

- Executa um bloco de código se uma condição for verdadeira



```
1  let idade = 18;  
2  
3  if (idade >= 18) {  
4      console.log("Você é maior de idade.");  
5  }
```

Estruturas de decisão (if/else)

- Executa um bloco de código se a condição for verdadeira e outro bloco se a condição for falsa.



```
1  let idade = 16;  
2  
3  if (idade >= 18) {  
4      console.log("Você é maior de idade.");  
5  } else {  
6      console.log("Você é menor de idade.");  
7  }
```

Estruturas de decisão (if/else if/else)

- Utilizada quando há várias condições a serem verificadas.



```
1  let nota = 85;
2
3  if (nota >= 90) {
4      console.log("A");
5  } else if (nota >= 80) {
6      console.log("B");
7  } else if (nota >= 70) {
8      console.log("C");
9  } else if (nota >= 60) {
10     console.log("D");
11 } else {
12     console.log("F");
13 }
14
```


Estruturas de decisão (switch)

- Executa um bloco de código quando existem várias condições possíveis para uma variável.

```
1  let dia = 3;
2  let diaSemana;
3
4  switch (dia) {
5      case 1:
6          diaSemana = "Domingo";
7          break;
8      case 2:
9          diaSemana = "Segunda-feira";
10         break;
11     case 3:
12         diaSemana = "Terça-feira";
13         break;
14     case 4:
15         diaSemana = "Quarta-feira";
16         break;
17     case 5:
18         diaSemana = "Quinta-feira";
19         break;
20     case 6:
21         diaSemana = "Sexta-feira";
22         break;
23     case 7:
24         diaSemana = "Sábado";
25         break;
26     default:
27         diaSemana = "Dia inválido";
28 }
29
30 console.log(diaSemana);
```

- Definem uma execução repetitiva de um bloco de código.
 - » for
 - » while
 - » do/while
 - » for/of
 - » for/in

Estruturas de repetição (for)

- Executa uma estrutura de bloco até que uma condição seja falsa;
- Utilizada quando se conhece o número de repetições desejado.

Estrutura:

```
for (expressão_inicial; condição; incremento) {  
    }  
}
```

- » expressão_inicial: declaração ou inicialização de contadores;
- » condição: condição de execução;
- » incremento: atualização de contadores. Executado ao final da estrutura de bloco.

Estruturas de repetição (for)

- Executa uma estrutura de bloco até que uma condição seja falsa;
- Utilizada quando se conhece o número de repetições desejado.



```
1  for (let contador = 0; contador < 5; contador++) {  
2      console.log("O valor do contador é " + contador);  
3  }
```

Estruturas de repetição (while)

- Executa uma estrutura de bloco enquanto uma determinada condição for verdadeira.



```
1  let contador = 0;  
2  
3  while (contador < 5) {  
4      console.log("O valor do contador é " + contador);  
5      contador++;  
6  }
```

Estruturas de repetição (do/while)

- A condição é analisada no final do bloco;
- Garante a execução do bloco ao menos uma vez.



```
1  let contador = 0;
2
3  do {
4      console.log("O valor do contador é " + contador);
5      contador++;
6  } while (contador < 5);
7
```

Estruturas de repetição (for/of)

- Itera sobre valores objetos iteráveis (arrays, strings, maps);
- Utilizado quando não se é necessário saber a posição (índice) de um determinado objeto.



```
1 let numeros = [1, 2, 3, 4, 5];  
2  
3 for (let numero of numeros) {  
4     console.log(numero);  
5 }
```

Estruturas de repetição (for/of)

- Itera sobre valores objetos iteráveis (arrays, strings, maps);
- Utilizado quando não se é necessário saber a posição (índice) de um determinado objeto.



```
1  let texto = "Hello";  
2  
3  for (let caractere of texto) {  
4      console.log(caractere);  
5  }
```


Estruturas de repetição (for/of)

- Itera sobre valores objetos iteráveis (arrays, strings, maps);
- Utilizado quando não se é necessário saber a posição (índice) de um determinado objeto.



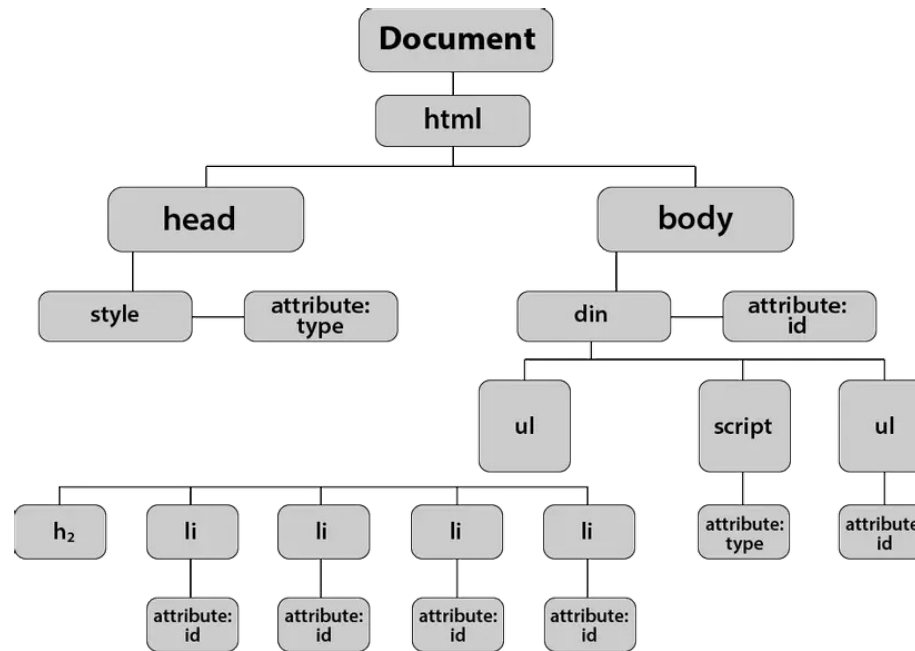
```
1  let mapa = new Map();
2  mapa.set('nome', 'João');
3  mapa.set('idade', 30);
4
5  for (let [chave, valor] of mapa) {
6      console.log(chave + ': ' + valor);
7  }
```

Estruturas de repetição (for/in)

- Itera sobre as chaves de um objeto.



```
1 let pessoa = { nome: "João", idade: 30, cidade: "Garanhuns" };  
2  
3 for (let propriedade in pessoa) {  
4     console.log(propriedade + ": " + pessoa[propriedade]);  
5 }  
6
```



- ***Document Object Model***;
- Representação estruturada de um documento HTML pelo *browser* para manipular e renderizar os elementos HTML;
- Representam todos os elementos como **objetos** em uma estrutura de dados denominado **árvore**.

- Javascript pode manipular os elementos do DOM, produzindo comportamento dinâmico a uma página;
- Através do objeto *document* é possível adicionar, remover ou modificar os elementos do DOM;



```
1 <body>
2   <p id="paragrafo"></p>
3   <script src="js/index.js"></script>
4 </body>
```



```
1 let p = document.getElementById("paragrafo");
2 p.textContent = "Olá mundo"
```

- *document.getElementById(id)*
 - » Obtém um elemento que possui um id específico
 - id: string
- *document.getElementsByClassName(classname)*
 - » Obtém uma lista de elemento que possui uma classe específica
 - classname: string
- *document.createElement(element)*
 - » Cria um elemento HTML
 - element: Nome do elemento HTML
- *element.appendChild(element)*
 - » Adiciona um elemento como filho
- *element.querySelector(selector)*
 - » Obtém o primeiro elemento, dado uma query
- *Element.querySelectorAll(selector)*
 - » Obtém uma lista com todos os elementos, dado uma query

Condicionais (if/else)

38



```
1  let idade = prompt("Entre com um número");  
2  
3  if (idade >= 18) {  
4      console.log("Você é maior de idade.");  
5  } else {  
6      console.log("Você é menor de idade.");  
7  }  
8
```

Condicionais (operador ternário)



```
1 let idade = prompt("Qual a sua idade?");  
2  
3 idade = +idade;  
4  
5 const resultado = document.getElementById("resultado");  
6 resultado.textContent = idade >= 18 ? "Sim" : "Não";
```

Condicionais (switch/case)

40

```
1 let dia = prompt("Qual o seu dia da semana preferido (entre com um número de 1 a 7)?");
2
3 dia = +dia;
4
5 switch (dia) {
6   case 1:
7     mensagem = "Domingo";
8     break;
9   case 2:
10    mensagem = "Segunda-feira";
11    break;
12   case 3:
13    mensagem = "Terça-feira";
14    break;
15   case 4:
16    mensagem = "Quarta-feira";
17    break;
18   case 5:
19    mensagem = "Quinta-feira";
20    break;
21   case 6:
22    mensagem = "Sexta-feira";
23    break;
24   case 7:
25    mensagem = "Sábado";
26    break;
27   default:
28    mensagem = "Dia inválido";
29 }
30
31 const p = document.getElementById("resultado");
32
33 p.textContent = "Dia preferido: " + mensagem
34 console.log("Dia preferido:" + mensagem);
```


Estrutura de repetição (while)

```
1 // Gera um número entre 1 e 10.
2 const numeroAleatorio = Math.floor(Math.random() * 10) + 1;
3
4 let suposicao;
5 let tentativas = 0;
6
7 do {
8   suposicao = prompt("Adivinhe o número entre 1 e 10:");
9   suposicao = +suposicao;
10
11   if (isNaN(suposicao) || suposicao < 1 || suposicao > 10) {
12     alert("Por favor, insira um número válido entre 1 e 10.");
13   } else {
14     tentativas++;
15     if (suposicao === numeroAleatorio) {
16       alert(
17         `Parabéns! Você adivinhou o número ${numeroAleatorio} em ${tentativas} tentativas.`
18       );
19       const div = document.getElementById("tentativas");
20       const p1 = document.createElement("p");
21       const p2 = document.createElement("p");
22
23       p1.textContent = "Número escolhido: " + numeroAleatorio;
24       p2.textContent = "Número de tentativas: " + tentativas;
25
26       div.appendChild(p1);
27       div.appendChild(p2);
28     } else if (suposicao < numeroAleatorio) {
29       alert("Tente um número maior.");
30     } else {
31       alert("Tente um número menor.");
32     }
33   }
34 } while (suposicao !== numeroAleatorio);
35
```

Estrutura de repetição (for)

42



```
1  let numero;  
2  let quantidade = 1;  
3  
4  const numeros = [];  
5  
6  do {  
7    numero = prompt(`Entre com o número ${quantidade}`);  
8    numero = +numero;  
9    if (isNaN(numero)) {  
10     alert("Entre apenas com números");  
11   } else {  
12     numeros.push(numero);  
13     quantidade++;  
14   }  
15 } while (quantidade <= 3);  
16  
17 const div = document.getElementById("resultado");  
18  
19 for (let x = 0; x < numeros.length; x++) {  
20   let p = document.createElement("p");  
21   p.textContent = numeros[x];  
22   div.appendChild(p);  
23 }
```



```
1  let numero;  
2  
3  do {  
4    numero = parseInt(prompt("Entre com um número:"));  
5  } while (isNaN(numero));  
6  
7  const p = document.getElementById("resultado");  
8  p.textContent = parOuImpar(numero);  
9  
10 function parOuImpar(num) {  
11   if (num % 2 === 0) {  
12     return "Par";  
13   } else {  
14     return "Ímpar";  
15   }  
16 }  
17
```

- Eventos: Ações que acontecem com um elemento HTML no navegador.
 - » Clicar de mouse;
 - » Pressionar um determinado botão;
 - » Carregar a página;
 - » Redimensionamento de uma janela.
- Ouvintes (*Listeners*): Funções responsáveis por detectar quando um determinado evento ocorre.
 - » Quando ocorrer, dispara uma ação (*callback*)



```
1  const meuElemento = document.getElementById("meuElemento");
2
3  function callbackF(valorDoEvento) {
4      // Ação a ser realizada quando um determinado evento ocorre
5  }
6
7  meuElemento.addEventListener("evento", callbackF);
8
```



```
1 meuElemento.addEventListener("evento", callbackF);
```

- Eventos de mouse:

- » click: Usuário clica em um elemento;
- » mouseover: Ocorre quando o cursor do mouse entra em um elemento;
- » mouseout: Ocorre quando o cursor do mouse sai de um elemento;
- » mousemove: Ocorre quando o cursor do mouse se move sobre um elemento;
- » mouseup: Ocorre quando o botão do mouse é pressionado;
- » mousedown: Ocorre quando o botão do mouse é liberado;



```
1 meuElemento.addEventListener("evento", callbackF);
```

- Eventos de teclado:

- » keydown: Ocorre quando uma tecla é pressionada
- » keyup: Ocorre quando uma tecla é liberada;
- » keypress: Ocorre quando uma tecla é pressionada e liberada;



```
1 meuElemento.addEventListener("evento", callbackF);
```

- Eventos de formulário:

- » submit: Ocorre quando o formulário é enviado;
- » input: Ocorre quando o valor de campo de entrada é alterado;
- » focus: Ocorre quando o elemento recebe foco;
- » blur: Ocorre quando o elemento perde foco.



```
1  function potencia(numero) {  
2      return numero ** 2;  
3  }  
4  
5  const potenciaV2 = function (numero) {  
6      return numero ** 2;  
7  }  
8  
9  const potenciaV3 = numero => numero ** 2;  
10  
11 console.log(potencia(2))  
12 console.log(potenciaV2(2))  
13 console.log(potenciaV3(2))
```



```
1  const meuElemento = document.getElementById("meuElemento");
2
3  // Alterar a cor de fundo para vermelho.
4  meuElemento.style.backgroundColor = "red";
5  meuElemento.style.color = "white";
6  // Removendo a cor de fundo
7  meuElemento.style.backgroundColor = "";
8
9
10 // Adicionar classes CSS
11 meuElemento.classList.add("nome-da-classe-CSS")
12 // Remover classes CSS
13 meuElemento.classList.remove("nome-da-classe-CSS")
```



```
1  const numeroAleatorio = Math.floor(Math.random() * 10) + 1;
2
3  // Obtém caixa e texto
4  const caixa = document.getElementById("caixa")
5  const resultado = document.getElementById("resultado")
6
7  resultado.textContent = numeroAleatorio;
8
9  function revelaNumero(valorDoEvento) {
10     console.log("Entrei na caixa")
11     valorDoEvento.target.style.backgroundColor = "#fff"
12 }
13
14 function escondeNumero(valorDoEvento) {
15     console.log("Sai da caixa")
16     valorDoEvento.target.style.backgroundColor = "#000"
17 }
18
19 caixa.addEventListener("mouseover", revelaNumero)
20 caixa.addEventListener("mouseout", escondeNumero)
```

- Implementação de um carrinho de compra ao site de ingressos

Ingressos disponíveis



O Som ao redor

O Som ao Redor começa com a chegada de uma milícia a uma rua de classe média da cidade do Recife, onde diferentes narrativas acabam se cruzando. Segundo a Associação Brasileira de Críticos de Cinema (Abraccine), é o 15º melhor filme da história do cinema nacional.

Ano: 2013
Direção: Kleber Mendonça
Gênero: Suspense, Drama
R\$ 3,50

[Adicionar ao carrinho](#)



Central do Brasil

Dora, uma ex-professora que escreve cartas na Central do Brasil, e o menino Josué, que fica órfão da noite para o dia. O filme, que emocionou o mundo, recebeu duas indicações ao Oscar: nas categorias melhor filme estrangeiro e melhor atriz.

Ano: 1998
Direção: Walter Salles
Gênero: Drama
R\$ 4,50

[Adicionar ao carrinho](#)



Cidade de Deus

Nos anos 1960, a favela é um complexo habitacional recém-construído longe do centro do Rio de Janeiro, com pouco acesso à eletricidade e água. Três ladrões amadores conhecidos como "Trio Ternura" — Cabeleira, Alicate e Marreco — aterrorizam os negócios locais. Marreco é o irmão de Buscapé. Como Robin Hood, eles dividem parte do dinheiro roubado com os habitantes da favela chamada de Cidade de Deus e, em troca, são protegidos por eles.

Ano: 2002
Direção: Fernando Meirelles
Gênero: Drama, Ação
R\$ 3,50

[Adicionar ao carrinho](#)



Tropa de Elite

Carrinho

R\$ 9,49

	Tropa de Elite	1	R\$ 4,99	x
	Central do Brasil	1	R\$ 4,50	x

Finalizar compra

- Representando ingressos via javascript;
- Renderizando ingressos:
 - » Definição de Ids;
 - » Definição das classes (posicionamento, espaçamento, cores);
 - » Adição dos *event listeners* dos botões de adicionar;
- Renderizando carrinho:
 - » Definição dos Ids;
 - » Definição das classes
 - » Adição dos *event listeners* dos botões de exclusão;
 - » Cálculo do total
 - » Habilitar/Desabilitar botão de *Finalizar compra*;

- Soluções lado cliente (*client side*) para armazenamento de dados de navegação;
- Soluções de armazenamento web:
 - » Local Storage
 - » Session Storage
 - » Cookies

- Local storage

- » Dados disponíveis mesmo após o navegador ser encerrado;
- » Acessíveis em todas as abas do mesmo domínio;
- » Armazena dados no formato chave-valor;
- » Navegadores definem um limite de 5MB;
- » Uso comum:
 - Preferências de usuário;
 - Carrinho de compras;
 - *Tokens* de autenticação.

Ex:

```
localStorage.getItem('carrinho')  
localStorage.setItem('carrinho', 'valor')  
localStorage.removeItem('carrinho')
```

- Session storage

- » Dados válidos apenas na sessão atual do navegador;
- » Excluídos após o navegador ser encerrado;
- » Armazena dados no formato chave-valor;
- » Acessíveis apenas na aba/janela onde foram definidos;
- » Uso comum:
 - Dados de formulários não enviados;
- » Ex:
 - `sessionStorage.getItem('carrinho')`
 - `sessionStorage.setItem('carrinho', 'valor')`
 - `sessionStorage.removeItem('carrinho')`

- Cookies

- » Método mais antigo de armazenamento de dados;
- » Encaminhado em toda requisição HTTP para o mesmo domínio;
- » Armazena dados como string;
- » Uso comum:
 - Rastreabilidade de usuário
 - Preferência de acesso
 - Análise de dados
 - Publicidade direcionada
- » Ex:
 - *`document.cookie = 'username=eduardo; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/';`*

- Capacidade de execução de código de forma **não bloqueante**, permitindo que outras tarefas continuem sendo executadas enquanto uma operação ainda não foi finalizada;
- Útil para lidar instruções de I/O:
 - » Chamadas de rede;
 - » Leitura de arquivos;
 - » Interações com banco de dados;
- Mecanismos de assincronismo
 - » *Callback*
 - » *Promises*
 - » *Async/Await*

- Funções passadas como argumento de outras funções que são executadas quando a operação assíncrona é concluída.



```
1 console.log("Início");
2
3 function executadaApos2s() {
4     console.log("Executando após 2 segundos");
5 }
6
7 setTimeout(executadaApos2s, 2000);
8
9 console.log("Fim");
```

Esta função é uma
callback

Callback hell

62

```

asyncFunc('something', function (err, data) {
  asyncFunc('something', function (err, data) {
    asyncFunc('something', function (err, data) {
      asyncFunc('something', function (err, data) {
        asyncFunc('something', function (err, data) {
          asyncFunc('something', function (err, data) {
            asyncFunc('something', function (err, data) {
              // do the final action
            });
          });
        });
      });
    });
  });
});

```



```
1  const axios = require("axios");
2
3  axios.get("https://jsonplaceholder.typicode.com/posts/1", function (response1) {
4    // Função de retorno de chamada 1
5    axios.get(
6      "https://jsonplaceholder.typicode.com/users/1",
7      function (response2) {
8        // Função de retorno de chamada 2
9        axios.get(
10         "https://jsonplaceholder.typicode.com/comments?postId=1",
11         function (response3) {
12           // Função de retorno de chamada 3
13           console.log("Post:", response1.data);
14           console.log("User:", response2.data);
15           console.log("Comments:", response3.data);
16         }
17       );
18     }
19   );
20 });
21
```

- Objeto Javascript que representa um valor no futuro (eventual conclusão ou falha de uma operação assíncrona);
- Utilizado para lidar com operações assíncronas (quando o resultado de uma operação ainda não é conhecido):
 - » Ex:
 - Solicitação de rede
 - Leitura de arquivo
 - Conversão de elementos
- Permite retornar de forma síncrona resultados assíncrono.
 - » Ex: Requisição na rede para uma página web que retorna um arquivo JSON.
 - Código lida de forma síncrona: a requisição retornará uma promessa para um JSON (*Promise<JSON>*)
 - Após recebido o JSON, a promessa é finalizada e tudo que depende diretamente do JSON é executado (*then*)
- Utilizado para evitar o *callback hell* (alinhamento excessivo de *callbacks*).



```
1  const fetchJson = (url) => {
2      return fetch(url)
3          .then((response) => {
4              if (!response.ok) {
5                  throw new Error(`Erro na solicitação: ${response.status}`);
6              }
7              return response.json();
8          });
9  };
10
11  fetchJson('https://jsonplaceholder.typicode.com/posts/1')
12      .then((data1) => {
13          console.log('Post:', data1);
14          return fetchJson('https://jsonplaceholder.typicode.com/users/1');
15      })
16      .then((data2) => {
17          console.log('User:', data2);
18          return fetchJson('https://jsonplaceholder.typicode.com/comments?postId=1');
19      })
20      .then((data3) => {
21          console.log('Comments:', data3);
22      })
23      .catch((error) => {
24          console.error(error);
25      });
```

• Criação

- » Objeto promise espera receber uma função com 2 argumentos
 - resolve: função que retorna um valor quando a operação assíncrona é bem sucedida.
 - reject: função que retorna um valor quando a operação assíncrona.
- » O argumento passado nas funções resolve ou reject serão utilizados na manipulação da promise.

```
1 let promessa = new Promise((resolve, reject) => {  
2   // Operação assíncrona  
3   let sucesso = true;  
4  
5   if (sucesso) {  
6     resolve("Operação bem-sucedida!");  
7   } else {  
8     reject("Operação falhou.");  
9   }  
10 });  
11
```

• Manipulação

- » Uma promise é manipulada através de dois métodos:
 - `then()`: executado quando a promise é bem-sucedida.
 - O argumento passado é uma função cujo argumento é o valor retornado pelo **resolve**.
 - `catch()`: executado quando a promise é mal-sucedida.
 - O argumento passado para `catch` é uma função cujo argumento é o valor retornado pelo **reject**.



```
1  promessa.then((mensagem) => {  
2    console.log(mensagem); // "Operação bem-sucedida!"  
3  }).catch((erro) => {  
4    console.log(erro); // "Operação falhou."  
5  });  
6
```

• Exemplo

- » Obter uma lista de posts do site:
<https://jsonplaceholder.typicode.com> e imprimir no console



```
1 fetch("https://jsonplaceholder.typicode.com" + "/posts").then((resposta) => {
2   if (!resposta.ok) {
3     throw new Error("A requisição não foi processada corretamente");
4   } else {
5     resposta.json().then((respostaJson) => {
6       console.log(respostaJson);
7     });
8   }
9 });
```

• Encadeamento de promises

- » Utilizado para tratar diversas funções assíncronas escritas de forma sequencial;
- » Uma promise pode retornar uma nova promise.
 - Uma sequência de *.then* encadeadas para tratar uma sequência de funções assíncronas.

```
1 fetch("https://jsonplaceholder.typicode.com" + "/posts")
2   .then((resposta) => {
3     if (!resposta.ok) {
4       throw new Error("A requisição não foi processada corretamente");
5     } else {
6       return resposta.json();
7     }
8   })
9   .then((respostaJson) => {
10     console.log(respostaJson);
11   });
```

Esta função retorna uma promise

Resolve a promise da função *fetch()*



```
1 fetch("https://jsonplaceholder.typicode.com" + "/posts")
2   .then((resposta) => {
3     if (!resposta.ok) {
4       throw new Error("A requisição não foi processada corretamente");
5     } else {
6       return resposta.json();
7     }
8   })
9   .then((respostaJson) => {
10     console.log(respostaJson);
11   });
```

Esta função retorna uma promise

Resolve a promise da função *json()*

• Propriedades

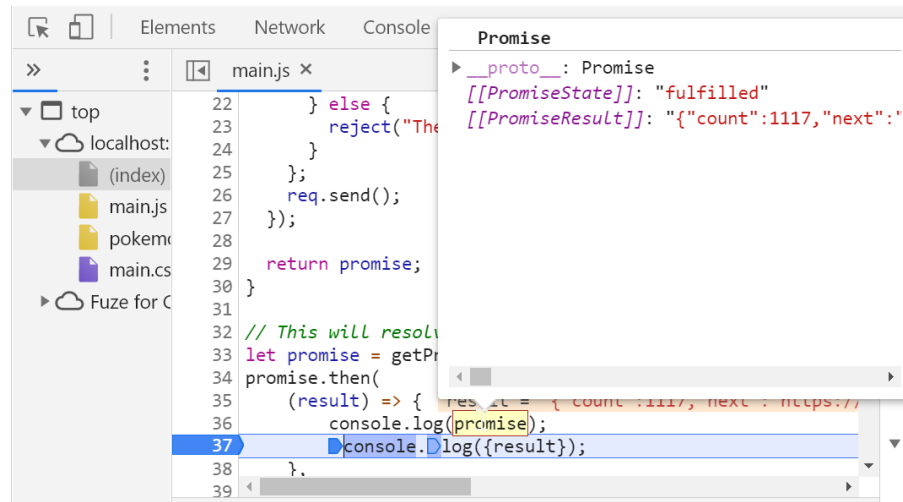
» Estados (state):

- Pendente (pending): Promise criada, mas não resolvida ou rejeitada;
- Cumprida (fulfilled): Operação assíncrona finalizada com sucesso;
- Rejeitada (reject): Operação assíncrona falhou.

» Resultado (result):

- undefined: quando o estado da promise é pending
- value: valor definido em resolve(value)
- error: valor definido em reject(error)

• Inacessíveis via código (apenas para depuração).



• Exemplo

- » No site <https://jsonplaceholder.typicode.com> Retornar a lista de todos os posts de um determinado usuário
 - Obter todos os posts
 - Obter o primeiro post e buscar informações do usuário
 - Buscar na lista de usuários todos os posts do mesmo


```
1  const API_URL = "https://jsonplaceholder.typicode.com";
2
3  // Função para buscar a lista de posts
4  function buscarPosts() {
5    return fetch(`${API_URL}/posts`).then((response) => {
6      if (!response.ok) {
7        throw new Error("Erro ao buscar posts");
8      }
9      return response.json();
10   });
11 }
12
13 // Função para buscar detalhes do usuário pelo ID do usuário
14 function buscarUsuario(userId) {
15   return fetch(`${API_URL}/users/${userId}`).then((response) => {
16     if (!response.ok) {
17       throw new Error("Erro ao buscar usuário");
18     }
19     return response.json();
20   });
21 }
22
23 // Função para buscar todos os posts de um usuário pelo ID do usuário
24 function buscarPostsPorUsuario(userId) {
25   return fetch(`${API_URL}/posts?userId=${userId}`).then((response) => {
26     if (!response.ok) {
27       throw new Error("Erro ao buscar posts do usuário");
28     }
29     return response.json();
30   });
31 }
32
33 // Encadeamento de Promises
34 buscarPosts()
35   .then((posts) => {
36     console.log("Posts:", posts);
37     const primeiroPost = posts[0];
38     return buscarUsuario(primeiroPost.userId);
39   })
40   .then((usuario) => {
41     console.log("Usuário:", usuario);
42     return buscarPostsPorUsuario(usuario.id);
43   })
44   .then((postsDoUsuario) => {
45     console.log("Posts do Usuário:", postsDoUsuario);
46   })
47   .catch((error) => {
48     console.error("Erro:", error);
49   });
```

- *Syntax sugar*
 - » Uma sintaxe simplificada para manipulação de promises.
- Permite escrever código assíncrono de forma sequencial;

Dúvidas?